

Clustering program

Generated by Doxygen 1.8.17

1 Modules Index	1
1.1 Modules List	1
2 Data Type Index	3
2.1 Data Types List	3
3 File Index	5
3.1 File List	5
4 Module Documentation	7
4.1 maths Module Reference	7
4.1.1 Function/Subroutine Documentation	7
4.1.1.1 calculate_cog()	7
4.1.1.2 calculate_distance()	8
4.1.1.3 cross()	8
4.1.1.4 rmsd()	8
4.1.1.5 update_complex()	9
4.1.1.6 vectors_angle_2d()	9
4.1.1.7 vectors_angle_3d()	10
4.2 mod_assoc Module Reference	10
4.2.1 Detailed Description	11
4.2.2 Function/Subroutine Documentation	11
4.2.2.1 allocate_assoc_object()	11
4.2.2.2 fill_assoc_object()	11
4.2.2.3 size_assoc()	12
4.3 mod_clust_algorithm Module Reference	12
4.3.1 Function/Subroutine Documentation	13
4.3.1.1 mean_linkage_clustering()	13
4.3.1.2 sort_complexes()	13
4.3.1.3 write_clust_info()	14
4.3.1.4 write_cluster_elements()	14
4.3.1.5 write_complexes()	15
4.4 mod_matrix Module Reference	15
4.4.1 Function/Subroutine Documentation	16
4.4.1.1 matrix_angle()	16
4.4.1.2 matrix_atoms_dist()	17
4.4.1.3 matrix_rmsd()	17
4.4.1.4 matrix_z_coord()	18
4.4.1.5 read_array()	18
4.4.1.6 read_matrix()	20
4.4.1.7 write_array()	20
4.4.1.8 write_matrix()	20
4.5 mod_pdb Module Reference	21

4.5.1 Detailed Description	21
4.5.2 Function/Subroutine Documentation	22
4.5.2.1 allocate_pdb_object()	22
4.5.2.2 count_atoms()	22
4.5.2.3 count_chains()	23
4.5.2.4 count_residues()	23
4.5.2.5 fill_pdb_object()	23
4.6 mod_threshold Module Reference	24
4.6.1 Function/Subroutine Documentation	24
4.6.1.1 array_angle()	24
4.6.1.2 array_atoms_dist()	25
4.6.1.3 array_z_coord()	25
4.6.1.4 sort_array()	26
4.7 read_input Module Reference	26
4.7.1 Function/Subroutine Documentation	26
4.7.1.1 read_assoc()	26
4.7.1.2 read_pdb()	27
5 Data Type Documentation	29
5.1 mod_assoc::type_assoc_file Type Reference	29
5.1.1 Detailed Description	29
5.1.2 Member Data Documentation	29
5.1.2.1 head	30
5.1.2.2 lines	30
5.1.2.3 nlines	30
5.1.2.4 rot1	30
5.1.2.5 rot2	30
5.1.2.6 trans_vector	30
5.1.2.7 xc1	30
5.1.2.8 xc2	31
5.2 mod_pdb::type_pdb_atom Type Reference	31
5.2.1 Member Data Documentation	31
5.2.1.1 beta	31
5.2.1.2 chainid	31
5.2.1.3 coord	31
5.2.1.4 id	32
5.2.1.5 name	32
5.2.1.6 occ	32
5.2.1.7 record	32
5.2.1.8 resid	32
5.2.1.9 resname	32
5.3 mod_pdb::type_pdb_chain Type Reference	32

5.4 mod_pdb::type_pdb_file Type Reference	33
5.4.1 Detailed Description	33
5.4.2 Member Data Documentation	33
5.4.2.1 atoms	33
5.4.2.2 chains	33
5.4.2.3 natoms	33
5.4.2.4 nchains	34
5.4.2.5 nresidues	34
5.4.2.6 residues	34
5.5 mod_pdb::type_pdb_residue Type Reference	34
5.5.1 Member Data Documentation	34
5.5.1.1 atoms	35
5.5.1.2 chainid	35
5.5.1.3 natoms	35
5.5.1.4 resid	35
5.5.1.5 resname	35
6 File Documentation	37
6.1 analyze_residues.f90 File Reference	37
6.1.1 Function/Subroutine Documentation	37
6.1.1.1 main()	37
6.1.1.2 print_help()	37
6.2 clust.f90 File Reference	38
6.2.1 Function/Subroutine Documentation	38
6.2.1.1 main()	38
6.2.1.2 print_help()	38
6.3 make_matrix.f90 File Reference	38
6.3.1 Function/Subroutine Documentation	38
6.3.1.1 main()	39
6.3.1.2 print_help()	39
6.3.1.3 read_atoms_coord()	39
6.4 maths.f90 File Reference	39
6.5 mod_assoc.f90 File Reference	40
6.6 mod_clust_algorithm.f90 File Reference	40
6.7 mod_matrix.f90 File Reference	41
6.8 mod_pdb.f90 File Reference	42
6.9 mod_threshold.f90 File Reference	42
6.10 read_input.f90 File Reference	43
6.11 threshold.f90 File Reference	43
6.11.1 Function/Subroutine Documentation	43
6.11.1.1 main()	43
6.11.1.2 print_help()	44

6.11.1.3 read_atoms_coord()	44
6.11.1.4 write_array()	44
6.11.1.5 write_complexes()	44

Index	45
--------------	-----------

Chapter 1

Modules Index

1.1 Modules List

Here is a list of all modules with brief descriptions:

maths	7
mod_assoc	
Association/complexes file utilities	10
mod_clust_algorithm	12
mod_matrix	15
mod_pdb	
Module to define a pdb type and pdb reading routine for tools	21
mod_threshold	24
read_input	26

Chapter 2

Data Type Index

2.1 Data Types List

Here are the data types with brief descriptions:

mod_assoc::type_assoc_file	
Data type for storing binned data (eg	29
mod_pdb::type_pdb_atom	31
mod_pdb::type_pdb_chain	32
mod_pdb::type_pdb_file	
Data type for storing binned data (eg	33
mod_pdb::type_pdb_residue	34

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

analyze_residues.f90	37
clust.f90	38
make_matrix.f90	38
maths.f90	39
mod_assoc.f90	40
mod_clust_algorithm.f90	40
mod_matrix.f90	41
mod_pdb.f90	42
mod_threshold.f90	42
read_input.f90	43
threshold.f90	43

Chapter 4

Module Documentation

4.1 maths Module Reference

Functions/Subroutines

- subroutine [cross](#) (v1, v2, v3)
Compute the cross product of two 3D vectors.
- subroutine [update_complex](#) (xc1, xc2, trans, rot_vx, rot_vy, nb_atoms, coords, new_coords)
Apply translation and rotation to a set of atomic coordinates.
- subroutine [vectors_angle_3d](#) (vector1, vector2, theta_degrees)
Compute the angle in degrees between two 3D vectors.
- subroutine [vectors_angle_2d](#) (vector1, vector2, theta_degrees)
Compute the angle in degrees between two vectors projected to 2D.
- subroutine [rmsd](#) (rmsd_value, nb_atoms, coords1, coords2)
Compute the root-mean-square deviation (RMSD) between two sets of coordinates.
- subroutine [calculate_cog](#) (cog, coords, nb_atoms)
Compute the center of geometry (centroid) of a set of points.
- subroutine [calculate_distance](#) (p1, p2, dist)
Compute Euclidean distance between two 3D points.

4.1.1 Function/Subroutine Documentation

4.1.1.1 [calculate_cog\(\)](#)

```
subroutine maths::calculate_cog (  
    real (kind=8), dimension(3), intent(out) cog,  
    real (kind=8), dimension(nb_atoms, 3), intent(in) coords,  
    integer, intent(in) nb_atoms )
```

Compute the center of geometry (centroid) of a set of points.

Sums the coordinates and divides by the number of points to produce the centroid.

Parameters

out	<i>cog</i>	Centroid vector (3 elements)
in	<i>coords</i>	Input coordinates, shape (nb_atoms,3)
in	<i>nb_atoms</i>	Number of atoms / points

4.1.1.2 calculate_distance()

```
subroutine maths::calculate_distance (
    real (kind=8), dimension(3), intent(in) p1,
    real (kind=8), dimension(3), intent(in) p2,
    real (kind=8), intent(out) dist )
```

Compute Euclidean distance between two 3D points.

Parameters

in	<i>p1</i>	First point (3 elements)
in	<i>p2</i>	Second point (3 elements)
out	<i>dist</i>	Euclidean distance $\ p1 - p2\ $

4.1.1.3 cross()

```
subroutine maths::cross (
    real ( kind=8 ), dimension ( 3 ), intent(in) v1,
    real ( kind=8 ), dimension ( 3 ), intent(in) v2,
    real ( kind=8 ), dimension ( 3 ), intent(out) v3 )
```

Compute the cross product of two 3D vectors.

Computes $v3 = v1 \times v2$ using the standard right-hand rule.

Parameters

in	<i>v1</i>	Left-hand operand vector (3 elements)
in	<i>v2</i>	Right-hand operand vector (3 elements)
out	<i>v3</i>	Resulting cross product (3 elements)

4.1.1.4 rmsd()

```
subroutine maths::rmsd (
    real(kind=8), intent(out) rmsd_value,
```

```
integer, intent(in) nb_atoms,
real(kind=8), dimension(:, :), intent(in) coords1,
real(kind=8), dimension(:, :), intent(in) coords2 )
```

Compute the root-mean-square deviation (RMSD) between two sets of coordinates.

Calculates $\text{RMSD} = \sqrt{(1/N) * \sum_i ||\text{coords1}(i,:) - \text{coords2}(i,:)||^2}$.

Parameters

out	<i>rmsd_value</i>	Resulting RMSD (scalar)
in	<i>nb_atoms</i>	Number of atoms / points (integer)
in	<i>coords1</i>	First coordinate set, shape (nb_atoms,3)
in	<i>coords2</i>	Second coordinate set, shape (nb_atoms,3)

4.1.1.5 update_complex()

```
subroutine maths::update_complex (
    real ( kind = 8 ), dimension ( 3 ), intent(in) xc1,
    real ( kind = 8 ), dimension ( 3 ), intent(in) xc2,
    real ( kind = 8 ), dimension ( 3 ), intent(in) trans,
    real ( kind = 8 ), dimension ( 3 ), intent(in) rot_vx,
    real ( kind = 8 ), dimension ( 3 ), intent(in) rot_vy,
    integer nb_atoms,
    real ( kind = 8 ), dimension (:, :), intent(in) coords,
    real ( kind = 8 ), dimension (:, :), intent(out) new_coords )
```

Apply translation and rotation to a set of atomic coordinates.

Transforms the coordinates of a complex from a frame centered at *xc2* to a new frame centered at *xc1*. The routine first recenters coordinates by *xc2*, applies a rotation defined by *rot_vx* and *rot_vy* (their cross product defines the third axis), then applies an additional translation *trans*, and writes the results to *new_coords*.

Parameters

in	<i>xc1</i>	Target center position (3)
in	<i>xc2</i>	Original center position (3)
in	<i>trans</i>	Translation to apply after rotation (3)
in	<i>rot_vx</i>	First rotation axis vector (3)
in	<i>rot_vy</i>	Second rotation axis vector (3)
in	<i>nb_atoms</i>	Number of atoms (integer)
in	<i>coords</i>	Input coordinates, shape (nb_atoms,3)
out	<i>new_coords</i>	Output coordinates, shape (nb_atoms,3)

4.1.1.6 vectors_angle_2d()

```
subroutine maths::vectors_angle_2d (
```

```

real( kind=8 ), dimension( 3 ), intent(in) vector1,
real( kind=8 ), dimension( 3 ), intent(in) vector2,
real (kind=8), intent(out) theta_degrees )

```

Compute the angle in degrees between two vectors projected to 2D.

Uses only the first two components of the input vectors to compute the angle in the XY plane. Returns 0 if either vector has zero length in the XY plane.

Parameters

in	<i>vector1</i>	First vector (use components 1 and 2)
in	<i>vector2</i>	Second vector (use components 1 and 2)
out	<i>theta_degrees</i>	Angle between projected vectors in degrees

4.1.1.7 vectors_angle_3d()

```

subroutine maths::vectors_angle_3d (
    real( kind=8 ), dimension( 3 ), intent(in) vector1,
    real( kind=8 ), dimension( 3 ), intent(in) vector2,
    real (kind=8), intent(out) theta_degrees )

```

Compute the angle in degrees between two 3D vectors.

Calculates the angle (in degrees) between *vector1* and *vector2* using the dot product formula and `acos`. No modification of inputs.

Parameters

in	<i>vector1</i>	First 3D vector (3 elements)
in	<i>vector2</i>	Second 3D vector (3 elements)
out	<i>theta_degrees</i>	Angle between vectors in degrees

4.2 mod_assoc Module Reference

Association/complexes file utilities.

Data Types

- type [type_assoc_file](#)

Data type for storing binned data (eg.

Functions/Subroutines

- subroutine `allocate_assoc_object` (this, nlines)
Allocate storage for an association/complexes object.
- integer function `size_assoc` (assoc_file)
Count encounter records in an association/complexes file.
- subroutine `fill_assoc_object` (this, assoc_file)
Read an association/complexes file and populate `type_assoc_file`.

4.2.1 Detailed Description

Association/complexes file utilities.

This module defines the `type_assoc_file` derived type and routines to allocate and populate it from an association/complexes text file. The `type_assoc_file` stores the raw file lines, header lines, per-encounter translation vectors and rotation vectors, and the two reference centers `xc1` and `xc2` extracted from the file header.

NOTE: The routines in this file are adapted from the SDA7 project's `mod_pdb.f90` (original author Neil Bruce). This file contains code and ideas derived from that implementation; appropriate attribution to the SDA7 sources is retained here. Consult the SDA7 project for original authorship and licensing details.

Author

Abraham Muñiz

4.2.2 Function/Subroutine Documentation

4.2.2.1 `allocate_assoc_object()`

```
subroutine mod_assoc::allocate_assoc_object (
    type(type_assoc_file), intent(inout) this,
    integer, intent(in) nlines )
```

Allocate storage for an association/complexes object.

Allocates the arrays inside `this` to hold `nlines` encounter records and initializes numeric arrays to zero.

Parameters

in, out	<i>this</i>	Instance of <code>type_assoc_file</code> to initialize
in	<i>nlines</i>	Number of encounter lines to allocate

4.2.2.2 `fill_assoc_object()`

```
subroutine mod_assoc::fill_assoc_object (
```

```
type(type_assoc_file), intent(inout) this,
integer, intent(in) assoc_file )
```

Read an association/complexes file and populate `type_assoc_file`.

Reads header lines from the open unit `assoc_file` and extracts the two reference centers (`xc1`, `xc2`) from header lines 3 and 4. Subsequent non-header lines are stored into `thislines` and the translation and rotation vectors are parsed into `thistrans_vector`, `thisrot1` and `thisrot2`.

Parameters

in, out	<code>this</code>	Allocated <code>type_assoc_file</code> to populate
in	<code>assoc_file</code>	Fortran unit number of an already-open file

4.2.2.3 size_assoc()

```
integer function mod_assoc::size_assoc (
    integer assoc_file )
```

Count encounter records in an association/complexes file.

Scans `assoc_file` (a Fortran unit number) and counts the number of non-comment lines. Lines starting with # are treated as comments and ignored. Returns the number of encounter records.

Parameters

in	<code>assoc_file</code>	Fortran unit number of an already-open file
----	-------------------------	---

Returns

`nlines` Number of encounter records found

4.3 mod_clust_algorithm Module Reference

Functions/Subroutines

- subroutine `mean_linkage_clustering` (`matrix`, `n`, `output_name`, `complexes`, `opt_array`)
Perform mean-linkage hierarchical clustering on a distance matrix.
- subroutine `sort_complexes` (`tot_encounters`, `cluster_indexes`, `cluster_count`, `active_clusters`, `representative_indexes`, `cluster_average`, `cluster_sd`, `array`)
Sort clusters and associated arrays by a given key array.
- subroutine `write_cluster_elements` (`tot_encounters`, `cluster_indexes`, `cluster_count`, `active_clusters`, `output_name`)
Write cluster membership lists to a text file.
- subroutine `write_clust_info` (`tot_encounters`, `representative_indexes`, `active_clusters`, `clust_count`, `cluster_average`, `cluster_sd`, `output_name`)
Write summary information about clusters to a text file.
- subroutine `write_complexes` (`tot_encounters`, `cluster_indexes`, `cluster_count`, `active_clusters`, `output_name`, `complexes`)
Write complexes contained in each cluster to individual files.

4.3.1 Function/Subroutine Documentation

4.3.1.1 mean_linkage_clustering()

```
subroutine mod_clust_algorithm::mean_linkage_clustering (
    real (kind=8), dimension(:, :), intent(inout) matrix,
    integer, intent(in) n,
    character*128, intent(in) output_name,
    type(type_assoc_file) complexes,
    real (kind=8), dimension(:), intent(inout), optional opt_array )
```

Perform mean-linkage hierarchical clustering on a distance matrix.

This routine performs average/mean linkage hierarchical clustering on the input `matrix` (shape (n,n)) of pairwise distances or similarity scores. The routine mutates `matrix` during merging and terminates merging when clusters are farther apart than an internally computed threshold. Results are written to files named using `output_name` and `complexes` information is optionally recorded via the `complexes` structure.

Parameters

in, out	<i>matrix</i>	Pairwise distance/similarity matrix (n,n). This matrix is updated during clustering.
in	<i>n</i>	Number of elements / encounters (integer).
in	<i>output_name</i>	Base name for output files (string).
in	<i>complexes</i>	Structure containing complex text lines/headers.
in	<i>opt_array</i>	Optional array of per-encounter values used to compute cluster averages instead of reading values from <code>matrix</code> .

4.3.1.2 sort_complexes()

```
subroutine mod_clust_algorithm::sort_complexes (
    integer, intent(in) tot_encounters,
    integer, dimension(:, :), intent(inout) cluster_indexes,
    integer, dimension(:), intent(inout) cluster_count,
    logical, dimension(:), intent(inout) active_clusters,
    integer, dimension(:), intent(inout) representative_indexes,
    real (kind=8), dimension(:), intent(inout) cluster_average,
    real (kind=8), dimension(:), intent(inout) cluster_sd,
    real (kind=8), dimension(:), intent(inout) array )
```

Sort clusters and associated arrays by a given key array.

Reorders `cluster_indexes` and all supplied per-cluster arrays so that they are sorted by ascending values in `array`. This is a stable reordering helper used prior to writing results.

Parameters

in	<i>tot_encounters</i>	Number of clusters/encounters
in, out	<i>cluster_indexes</i>	Matrix storing cluster member indexes

Parameters

in, out	<i>cluster_count</i>	Number of members in each cluster
in, out	<i>active_clusters</i>	Logical flags marking active clusters
in, out	<i>representative_indexes</i>	Representative index per cluster
in, out	<i>cluster_average</i>	Per-cluster average values
in, out	<i>cluster_sd</i>	Per-cluster standard deviation
in, out	<i>array</i>	Key array used to sort clusters

4.3.1.3 write_clust_info()

```

subroutine mod_clust_algorithm::write_clust_info (
    integer, intent(in) tot_encounters,
    integer, dimension(:), intent(in) representative_indexes,
    logical, dimension(:), intent(in) active_clusters,
    integer, dimension(tot_encounters), intent(in) clust_count,
    real (kind=8), dimension(tot_encounters) cluster_average,
    real (kind=8), dimension(tot_encounters) cluster_sd,
    character*128, intent(in) output_name )

```

Write summary information about clusters to a text file.

Produces a summary file <output_name>_clust_info.txt listing cluster population, representative index, average and standard deviation for each active cluster.

Parameters

in	<i>tot_encounters</i>	Total number of encounters
in	<i>clust_count</i>	Array with population of each cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>representative_indexes</i>	Representative index per cluster
in	<i>output_name</i>	Base filename for outputs
in	<i>cluster_average</i>	Per-cluster average values
in	<i>cluster_sd</i>	Per-cluster standard deviations

4.3.1.4 write_cluster_elements()

```

subroutine mod_clust_algorithm::write_cluster_elements (
    integer, intent(in) tot_encounters,
    integer, dimension(:, :), intent(in) cluster_indexes,
    integer, dimension(:), intent(in) cluster_count,
    logical, dimension(:), intent(in) active_clusters,
    character*128, intent(in) output_name )

```

Write cluster membership lists to a text file.

Outputs a file named <output_name>_clusters.txt containing the members of each active cluster in a matrix-like format.

Parameters

in	<i>tot_encounters</i>	Number of possible clusters/encounters
in	<i>cluster_indexes</i>	Matrix of cluster member indexes
in	<i>cluster_count</i>	Number of members per cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>output_name</i>	Base filename for output

4.3.1.5 write_complexes()

```
subroutine mod_clust_algorithm::write_complexes (
    integer, intent(in) tot_encounters,
    integer, dimension(:, :), intent(in) cluster_indexes,
    integer, dimension(:), intent(in) cluster_count,
    logical, dimension(:), intent(in) active_clusters,
    character*128, intent(in) output_name,
    type(type_assoc_file) complexes )
```

Write complexes contained in each cluster to individual files.

For each active cluster this routine creates a file <output_name>_clusterXXXX_complexes containing the header lines from complexeshead and the encounter lines for the cluster members taken from complexeslines.

Parameters

in	<i>tot_encounters</i>	Number of clusters/encounters
in	<i>cluster_indexes</i>	Matrix of cluster member indexes
in	<i>cluster_count</i>	Number of members per cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>output_name</i>	Base filename for output files
in	<i>complexes</i>	Structure with header and lines to write

4.4 mod_matrix Module Reference

Functions/Subroutines

- subroutine [matrix_z_coord](#) (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Build a matrix of absolute differences of Z-coordinates per encounter.
- subroutine [matrix_atoms_dist](#) (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Build matrix from minimum inter-atomic distances between two solutes.
- subroutine [matrix_angle](#) (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Build matrix of angular differences between encounters.
- subroutine [matrix_rmsd](#) (matrix, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, coord)

Build RMSD-based pairwise matrix between transformed coordinates.

- subroutine `write_matrix` (matrix, filename)

Write a numeric matrix to a formatted text file.

- subroutine `write_array` (array, filename)

Write a numeric array to a formatted text file.

- subroutine `read_matrix` (matrix, n, filename)

Read a square numeric matrix from a formatted text file.

- subroutine `read_array` (array, n, filename)

Read a numeric array from a formatted text file.

4.4.1 Function/Subroutine Documentation

4.4.1.1 `matrix_angle()`

```
subroutine mod_matrix::matrix_angle (
    real(kind=8), dimension(n, n), intent(out) matrix,
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:), intent(in) point1a,
    real(kind=8), dimension(:), intent(in) point1b,
    real(kind=8), dimension(:), intent(in) point2a,
    real(kind=8), dimension(:), intent(in) point2b,
    integer, intent(in) dimensions )
```

Build matrix of angular differences between encounters.

Computes the angle (2D or 3D) for each encounter and fills the pairwise absolute difference matrix.

Parameters

out	<i>matrix</i>	Output (n,n) matrix of pairwise angular differences
out	<i>array</i>	Output (n) array with per-encounter angles
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in solute 2 to transform
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Transform translation vectors (n,3)
in	<i>rot1,rot2</i>	Transform rotation arrays (n,3)
in	<i>point1a</i>	First point defining reference vector
in	<i>point1b</i>	Second point defining reference vector
in	<i>point2a</i>	First point of vector to be transformed
in	<i>point2b</i>	Second point of vector to be transformed
in	<i>dimensions</i>	Dimensionality of angle calculation (2 or 3)

4.4.1.2 matrix_atoms_dist()

```

subroutine mod_matrix::matrix_atoms_dist (
    real(kind=8), dimension(n, n), intent(out) matrix,
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute1_crds,
    real(kind=8), dimension(:, :), intent(in) solute2_crds )

```

Build matrix from minimum inter-atomic distances between two solutes.

For each encounter, the second solute is transformed and the minimum distance to `solute1_crds` is computed; pairwise absolute differences of these minima populate `matrix`.

Parameters

out	<i>matrix</i>	Output (n,n) matrix of pairwise values
out	<i>array</i>	Output (n) array with per-encounter minimum distances
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in solute 2 to transform
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Transform translation vectors (n,3)
in	<i>rot1,rot2</i>	Transform rotation arrays (n,3)
in	<i>solute1_crds</i>	Coordinates of first solute atoms
in	<i>solute2_crds</i>	Coordinates of second solute atoms

4.4.1.3 matrix_rmsd()

```

subroutine mod_matrix::matrix_rmsd (
    real(kind=8), dimension(n, n), intent(out) matrix,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) coord )

```

Build RMSD-based pairwise matrix between transformed coordinates.

Applies transformation to the coordinate set for each encounter and computes pairwise RMSD values between transformed coordinate sets.

Parameters

out	<i>matrix</i>	Output (n,n) matrix of pairwise RMSD values
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in solute 2 to transform
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Transform translation vectors (n,3)
in	<i>rot1,rot2</i>	Transform rotation arrays (n,3)
in	<i>coord</i>	Coordinates to be transformed

4.4.1.4 **matrix_z_coord()**

```

subroutine mod_matrix::matrix_z_coord (
    real(kind=8), dimension(n, n), intent(out) matrix,
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute_crds )

```

Build a matrix of absolute differences of Z-coordinates per encounter.

Computes the Z coordinate for each encounter (as in `array_z_coord`) and fills `matrix(i, j) = |z_i - z_j|` for $i < j$ (symmetrically).

Parameters

out	<i>matrix</i>	Output (n,n) matrix of pairwise values
out	<i>array</i>	Output array with per-encounter Z values
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <code>solute_crds</code>
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector,rot1,rot2</i>	Transform arrays (n,3)
in	<i>solute_crds</i>	Coordinates of solute atoms

4.4.1.5 **read_array()**

```

subroutine mod_matrix::read_array (
    real(kind=8), dimension(:, :), intent(out), allocatable array,
    integer, intent(inout) n,
    character*128, intent(in) filename )

```

Read a numeric array from a formatted text file.

Parses a single-row formatted array file produced by `write_array`.

Parameters

out	<i>array</i>	Output array read from file (allocated)
in, out	<i>n</i>	Number of encounters (adjusted if file is smaller)
in	<i>filename</i>	Input filename

4.4.1.6 read_matrix()

```

subroutine mod_matrix::read_matrix (
    real(kind=8), dimension(:, :), intent(out), allocatable matrix,
    integer, intent(inout) n,
    character*128, intent(in) filename )

```

Read a square numeric matrix from a formatted text file.

Reads the file line-by-line to determine matrix size and parses rows with the expected F10.4 formatting.

Parameters

out	<i>matrix</i>	Output matrix read from file (allocated)
in, out	<i>n</i>	Number of encounters (adjusted if file is smaller)
in	<i>filename</i>	Input filename

4.4.1.7 write_array()

```

subroutine mod_matrix::write_array (
    real(kind=8), dimension(:), intent(in) array,
    character*128, intent(in) filename )

```

Write a numeric array to a formatted text file.

Writes *array* as a single formatted line using F10.4 floats.

Parameters

in	<i>array</i>	Input array to write
in	<i>filename</i>	Output filename

4.4.1.8 write_matrix()

```

subroutine mod_matrix::write_matrix (
    real(kind=8), dimension(:, :), intent(in) matrix,
    character*128, intent(in) filename )

```

Write a numeric matrix to a formatted text file.

Writes `matrix` with each row as `F10.4` formatted floats. The output filename is provided by `filename`.

Parameters

in	<code>matrix</code>	Input matrix to write
in	<code>filename</code>	Output filename

4.5 mod_pdb Module Reference

Module to define a pdb type and pdb reading routine for tools.

Data Types

- type `type_pdb_atom`
- type `type_pdb_chain`
- type `type_pdb_file`
Data type for storing binned data (eg.
- type `type_pdb_residue`

Functions/Subroutines

- subroutine `allocate_pdb_object` (`this`, `pdb_file`, `natoms`, `nresidues`, `nchains`)
Allocate and initialize a `type_pdb_file` structure from a PDB file.
- integer function `count_atoms` (`pdb_file`)
Count ATOM/HETATM records in a PDB file.
- integer function `count_residues` (`pdb_file`)
Count residues in a PDB file.
- integer function `count_chains` (`pdb_file`)
Count chain separators (TER records) in a PDB file.
- subroutine `fill_pdb_object` (`this`, `pdb_file`)
Parse PDB records and populate the `type_pdb_file` structure.

4.5.1 Detailed Description

Module to define a pdb type and pdb reading routine for tools.

NOTE: This implementation is based on the SDA7 project's `mod_pdb.f90` (original author Neil Bruce). The present file has been modified and expanded for the clustering tools in this workspace. Notable enhancements include the addition of `count_residues` and `count_chains` functions and changes that allow creation of richer in-memory objects where residues contain their atoms and chains contain residues and atoms. These changes make it possible to build `type_pdb_file` instances with proper `residues` and `chains` substructures (each storing their atoms) rather than storing atoms in a single flat list only.

The original SDA7 code and author attribution are retained; consult the SDA7 sources for original licensing and authorship details.

Author

Neil Bruce @modified-by Abraham Muñiz Chicharro

4.5.2 Function/Subroutine Documentation

4.5.2.1 allocate_pdb_object()

```
subroutine mod_pdb::allocate_pdb_object (
    type(type_pdb_file), intent(inout) this,
    integer, intent(in) pdb_file,
    integer, intent(in) natoms,
    integer, intent(in) nresidues,
    integer, intent(in) nchains )
```

Allocate and initialize a `type_pdb_file` structure from a PDB file.

This routine allocates the storage for `thisatoms`, `thisresidues` and `thischains` using the supplied counts and performs a preliminary scan of the open Fortran unit `pdb_file` to determine per-chain and per-residue allocation sizes for nested arrays.

Parameters

in, out	<i>this</i>	<code>type_pdb_file</code> instance to allocate and initialize
in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
in	<i>natoms</i>	Total number of ATOM/HETATM records expected
in	<i>nresidues</i>	Number of residues expected
in	<i>nchains</i>	Number of chains expected

4.5.2.2 count_atoms()

```
integer function mod_pdb::count_atoms (
    integer pdb_file )
```

Count ATOM/HETATM records in a PDB file.

Scans the open Fortran file unit `pdb_file` and returns the number of lines starting with ATOM or HETATM.

Parameters

in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
----	-----------------	---

Returns

`natoms` Number of ATOM/HETATM records found

4.5.2.3 count_chains()

```
integer function mod_pdb::count_chains (
    integer pdb_file )
```

Count chain separators (TER records) in a PDB file.

Scans the open Fortran file unit `pdb_file` and counts the number of TER records, which are treated as chain separators. Returns the number of chains found.

Parameters

in	<code>pdb_file</code>	Fortran unit number of an already-open PDB file
----	-----------------------	---

Returns

`nchains` Number of chains (TER records) found

4.5.2.4 count_residues()

```
integer function mod_pdb::count_residues (
    integer pdb_file )
```

Count residues in a PDB file.

Scans the open Fortran file unit `pdb_file` and counts residue changes by comparing residue identifiers in AT↔OM/HETATM records. Returns the number of distinct residues found.

Parameters

in	<code>pdb_file</code>	Fortran unit number of an already-open PDB file
----	-----------------------	---

Returns

`nresidues` Number of residues found

4.5.2.5 fill_pdb_object()

```
subroutine mod_pdb::fill_pdb_object (
    type(type_pdb_file), intent(inout) this,
    integer, intent(in) pdb_file )
```

Parse PDB records and populate the `type_pdb_file` structure.

Reads ATOM/HETATM records from the already-open Fortran unit `pdb_file`, fills `thisatoms` with atom records and populates `thisresidues` and `thischains` so that residues contain their per-residue atoms arrays and chains contain their residues and atoms. The routine expects `this` to have been allocated with the correct sizes (see `allocate_pdb_object`).

Parameters

in, out	<i>this</i>	Pre-allocated <code>type_pdb_file</code> to populate
in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file

4.6 mod_threshold Module Reference

Functions/Subroutines

- subroutine `array_z_coord` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter.
- subroutine `array_atoms_dist` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter.
- subroutine `array_angle` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter.
- subroutine `sort_array` (arr, encounter_indexes)
Sort an array in ascending order and reorder encounter indexes accordingly.

4.6.1 Function/Subroutine Documentation

4.6.1.1 array_angle()

```

subroutine mod_threshold::array_angle (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:), intent(in) point1a,
    real(kind=8), dimension(:), intent(in) point1b,
    real(kind=8), dimension(:), intent(in) point2a,
    real(kind=8), dimension(:), intent(in) point2b,
    integer, intent(in) dimensions )

```

Compute angle between two vectors (defined by atom groups) per encounter.

Transforms the pair of points from solute2 for each encounter and computes the angle between the vector defined by point1b–point1a and the vector defined by the transformed point2b–point2a.

Parameters

out	<i>array</i>	Output array with angles in degrees (n)
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms used for transformation
in	<i>xc1,xc2</i>	Centers used for transform
in	<i>trans_vector,rot1,rot2</i>	Transform arrays (n,3)
in	<i>point1a,b.point2a,b</i>	Reference points/vectors

4.6.1.2 array_atoms_dist()

```

subroutine mod_threshold::array_atoms_dist (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute1_crds,
    real(kind=8), dimension(:, :), intent(in) solute2_crds )

```

Compute minimum inter-atomic distance between two solutes for each encounter.

For each encounter the second solute is transformed and the minimum distance between any atom of `solute1_crds` and the transformed `solute2_crds` is computed and stored in `array`.

Parameters

out	<i>array</i>	Output array (n)
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <code>solute2_crds</code>
in	<i>xc1,xc2</i>	Centers used for transform
in	<i>trans_vector,rot1,rot2</i>	Transform arrays (n,3)
in	<i>solute1_crds</i>	Coordinates of solute1 atoms
in	<i>solute2_crds</i>	Coordinates of solute2 atoms

4.6.1.3 array_z_coord()

```

subroutine mod_threshold::array_z_coord (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute_crds )

```

Compute the Z-coordinate (along transformed z-axis) for each encounter.

Recenters each encounter at `xc2`, applies rotation/translation using `rot1/rot2` and `trans_vector`, and extracts the Z coordinate of the first atom in `solute_crds` for every encounter into `array`.

Parameters

out	<i>array</i>	Output array of length <i>n</i> with z-coordinates
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <i>solute_crds</i>
in	<i>xc1,xc2</i>	Centers used for transform
in	<i>trans_vector,rot1,rot2</i>	Transform arrays (n,3)
in	<i>solute_crds</i>	Coordinates of solute atoms (nb_atoms,3)

4.6.1.4 sort_array()

```
subroutine mod_threshold::sort_array (
    real (kind=8), dimension(:), intent(inout) arr,
    integer, dimension(:), intent(inout) encounter_indexes )
```

Sort an array in ascending order and reorder encounter indexes accordingly.

Simple selection sort used to reorder *arr* and the parallel *encounter_indexes* array. Intended for small to moderate sizes.

Parameters

in, out	<i>arr</i>	Array to sort (modified in-place)
in, out	<i>encounter_indexes</i>	Parallel index array reordered to match <i>arr</i>

4.7 read_input Module Reference

Functions/Subroutines

- subroutine [read_pdb](#) (pdb, filename, tot_atoms, tot_residues, tot_chains)
Read a PDB file into a type_pdb_file object.
- subroutine [read_assoc](#) (assoc, filename, tot_encounters)
Read an association/complexes file into a type_assoc_file object.

4.7.1 Function/Subroutine Documentation

4.7.1.1 read_assoc()

```
subroutine read_input::read_assoc (
    type ( type_assoc_file ) assoc,
    character*128, intent(in) filename,
    integer, intent(out) tot_encounters )
```

Read an association/complexes file into a *type_assoc_file* object.

Checks the file exists, opens it and uses *size_assoc*, *allocate_assoc_object* and *fill_assoc_↔* object to construct the *assoc* structure.

Parameters

out	<i>assoc</i>	Allocated and filled assoc object
in	<i>filename</i>	Path to complexes file
out	<i>tot_encounters</i>	Number of encounter records read

4.7.1.2 read_pdb()

```
subroutine read_input::read_pdb (
    type ( type_pdb_file ) pdb,
    character*128, intent(in) filename,
    integer, intent(out) tot_atoms,
    integer, intent(out) tot_residues,
    integer, intent(out) tot_chains )
```

Read a PDB file into a `type_pdb_file` object.

Validates the existence of `filename`, opens it, determines the counts for atoms, residues and chains, allocates `pdb` and fills it using `allocate_pdb_object` and `fill_pdb_object`.

Parameters

out	<i>pdb</i>	Allocated and filled PDB object
in	<i>filename</i>	Path to PDB file
out	<i>tot_atoms</i>	Number of atoms found
out	<i>tot_residues</i>	Number of residues found
out	<i>tot_chains</i>	Number of chains found

Chapter 5

Data Type Documentation

5.1 `mod_assoc::type_assoc_file` Type Reference

Data type for storing binned data (eg.

Public Attributes

- integer `nlines`
Number of lines in assoc_file file.
- character(len=217), dimension(:), allocatable `lines`
- character(len=217), dimension(4) `head`
- real(kind=8), dimension(:, :), allocatable `trans_vector`
Translational vector.
- real(kind=8), dimension(:, :), allocatable `rot1`
rotX, rotY
- real(kind=8), dimension(:, :), allocatable `rot2`
- real(kind=8), dimension(3) `xc1`
center of mass of p2
- real(kind=8), dimension(3) `xc2`

5.1.1 Detailed Description

Data type for storing binned data (eg.

Radial distribution functions)

5.1.2 Member Data Documentation

5.1.2.1 head

```
character(len=217), dimension(4) mod_assoc::type_assoc_file::head
```

5.1.2.2 lines

```
character(len=217), dimension(:), allocatable mod_assoc::type_assoc_file::lines
```

5.1.2.3 nlines

```
integer mod_assoc::type_assoc_file::nlines
```

Number of lines in assoc_file file.

5.1.2.4 rot1

```
real (kind=8), dimension (:,:), allocatable mod_assoc::type_assoc_file::rot1
```

rotX, rotY

5.1.2.5 rot2

```
real (kind=8), dimension (:,:), allocatable mod_assoc::type_assoc_file::rot2
```

5.1.2.6 trans_vector

```
real (kind=8), dimension (:, :), allocatable mod_assoc::type_assoc_file::trans_vector
```

Translational vector.

5.1.2.7 xc1

```
real (kind=8), dimension (3) mod_assoc::type_assoc_file::xc1
```

center of mass of p2

5.1.2.8 xc2

```
real (kind=8), dimension (3) mod_assoc::type_assoc_file::xc2
```

The documentation for this type was generated from the following file:

- [mod_assoc.f90](#)

5.2 mod_pdb::type_pdb_atom Type Reference

Public Attributes

- integer [record](#)
Number of atoms in pdb file Flag for record type, 1 = ATOM, 2 = HETATM.
- real(kind=8), dimension(3) [coord](#)
Position, occupancy and temp factor - can be used to store other data eg in pqr.
- real(kind=8) [occ](#)
- real(kind=8) [beta](#)
- character(len=4) [name](#)
Atom name, res name and chain name.
- character(len=3) [resname](#)
- character(len=1) [chainid](#)
- integer [id](#)
Atom and residue numbers.
- integer [resid](#)

5.2.1 Member Data Documentation

5.2.1.1 beta

```
real (kind=8) mod_pdb::type_pdb_atom::beta
```

5.2.1.2 chainid

```
character (len=1) mod_pdb::type_pdb_atom::chainid
```

5.2.1.3 coord

```
real (kind=8), dimension (3) mod_pdb::type_pdb_atom::coord
```

Position, occupancy and temp factor - can be used to store other data eg in pqr.

5.2.1.4 id

```
integer mod_pdb::type_pdb_atom::id
```

Atom and residue numbers.

5.2.1.5 name

```
character (len=4) mod_pdb::type_pdb_atom::name
```

Atom name, res name and chain name.

5.2.1.6 occ

```
real (kind=8) mod_pdb::type_pdb_atom::occ
```

5.2.1.7 record

```
integer mod_pdb::type_pdb_atom::record
```

Number of atoms in pdb file Flag for record type, 1 = ATOM, 2 = HETATM.

5.2.1.8 resid

```
integer mod_pdb::type_pdb_atom::resid
```

5.2.1.9 resname

```
character (len=3) mod_pdb::type_pdb_atom::resname
```

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

5.3 mod_pdb::type_pdb_chain Type Reference

Collaboration diagram for mod_pdb::type_pdb_chain:

5.4 mod_pdb::type_pdb_file Type Reference

Data type for storing binned data (eg.

Collaboration diagram for mod_pdb::type_pdb_file:

Public Attributes

- integer [natoms](#)
Number of atoms in pdb file.
- integer [nresidues](#)
- integer [nchains](#)
- type([type_pdb_atom](#)), dimension(:), allocatable [atoms](#)
Flag for record type, 1 = ATOM, 2 = HETATM.
- type([type_pdb_residue](#)), dimension(:), allocatable [residues](#)
- type([type_pdb_chain](#)), dimension(:), allocatable [chains](#)

5.4.1 Detailed Description

Data type for storing binned data (eg.

Radial distribution functions)

5.4.2 Member Data Documentation

5.4.2.1 atoms

```
type(type\_pdb\_atom), dimension (:), allocatable mod_pdb::type_pdb_file::atoms
```

Flag for record type, 1 = ATOM, 2 = HETATM.

5.4.2.2 chains

```
type(type\_pdb\_chain), dimension (:), allocatable mod_pdb::type_pdb_file::chains
```

5.4.2.3 natoms

```
integer mod_pdb::type_pdb_file::natoms
```

Number of atoms in pdb file.

5.4.2.4 nchains

```
integer mod_pdb::type_pdb_file::nchains
```

5.4.2.5 nresidues

```
integer mod_pdb::type_pdb_file::nresidues
```

5.4.2.6 residues

```
type(type_pdb_residue), dimension (:), allocatable mod_pdb::type_pdb_file::residues
```

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

5.5 mod_pdb::type_pdb_residue Type Reference

Collaboration diagram for mod_pdb::type_pdb_residue:

Public Attributes

- integer [natoms](#)
Number of atoms in pdb file.
- character [chainid](#)
Chain id.
- character(len=3) [resname](#)
Residue name.
- integer [resid](#)
Residue number.
- type([type_pdb_atom](#)), dimension(:), allocatable [atoms](#)
Flag for record type, 1 = ATOM, 2 = HETATM.

5.5.1 Member Data Documentation

5.5.1.1 atoms

`type(type_pdb_atom), dimension (:), allocatable mod_pdb::type_pdb_residue::atoms`

Flag for record type, 1 = ATOM, 2 = HETATM.

5.5.1.2 chainid

`character mod_pdb::type_pdb_residue::chainid`

Chain id.

5.5.1.3 natoms

`integer mod_pdb::type_pdb_residue::natoms`

Number of atoms in pdb file.

5.5.1.4 resid

`integer mod_pdb::type_pdb_residue::resid`

Residue number.

5.5.1.5 resname

`character (len=3) mod_pdb::type_pdb_residue::resname`

Residue name.

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

Chapter 6

File Documentation

6.1 analyze_residues.f90 File Reference

Functions/Subroutines

- program `main`
Analyze which residues are close to the surface across encounters.
- subroutine `print_help` ()
Print usage/help text for `analyze_residues`.

6.1.1 Function/Subroutine Documentation

6.1.1.1 `main()`

```
program main
```

Analyze which residues are close to the surface across encounters.

This program reads a PDB for solute2 and an associations/complexes file, transforms residue centers for each encounter and counts how many encounters bring residues within a distance threshold from the surface. Results are written to `closest_residues.txt`.

6.1.1.2 `print_help()`

```
subroutine main::print_help
```

Print usage/help text for `analyze_residues`.

Shows required command-line options and examples for running the residue analysis tool.

6.2 clust.f90 File Reference

Functions/Subroutines

- program [main](#)
Command-line program to perform hierarchical clustering on encounter matrices.
- subroutine [print_help](#) ()
Print usage and help information for the `clust` program.

6.2.1 Function/Subroutine Documentation

6.2.1.1 `main()`

`program main`

Command-line program to perform hierarchical clustering on encounter matrices.

Reads a pairwise matrix (and optional array) and runs mean-linkage hierarchical clustering. Results are written to files using the `-output_name` base name. See `print_help` below for usage.

6.2.1.2 `print_help()`

`subroutine main::print_help`

Print usage and help information for the `clust` program.

Describes command-line options and provides an example invocation.

6.3 make_matrix.f90 File Reference

Functions/Subroutines

- program [main](#)
Command-line utility to build encounter-based matrices.
- subroutine [print_help](#) (help_option)
Print usage/help text for `make_matrix`.
- subroutine [read_atoms_coord](#) (atoms_array, atoms_coord, tot_atoms, pdb, pdb_filename)
Read coordinates for a specified list of atoms (names or indices).

6.3.1 Function/Subroutine Documentation

6.3.1.1 main()

```
program main
```

Command-line utility to build encounter-based matrices.

Usage: see `print_help` below. This program reads PDB(s) and an association/complex file and generates matrices (`rmsd`, `z_coord`, `atoms_dist`, `angle`) and arrays derived from encounter transforms.

6.3.1.2 print_help()

```
subroutine main::print_help (
    character(len=*), intent(in) help_option )
```

Print usage/help text for `make_matrix`.

Provides short usage examples for the main program and for each supported matrix type.

6.3.1.3 read_atoms_coord()

```
subroutine main::read_atoms_coord (
    character(len=*), dimension(:), intent(in) atoms_array,
    real(kind=8), dimension(:, :), intent(out), allocatable atoms_coord,
    integer, intent(in) tot_atoms,
    type ( type_pdb_file ), intent(in) pdb,
    character(len=*), intent(in) pdb_filename )
```

Read coordinates for a specified list of atoms (names or indices).

Accepts either atom indices or atom names. If atom names are provided, all matching atoms in the PDB are returned (in order).

Parameters

in	<code>atoms_array</code>	Array of atom name strings or index strings
in	<code>pdb_filename</code>	Reference filename for warnings
in	<code>tot_atoms</code>	Total number of atoms in <code>pdb</code>
out	<code>atoms_coord</code>	Allocated array with atom coordinates (N,3)

6.4 maths.f90 File Reference

Modules

- module [maths](#)

Functions/Subroutines

- subroutine [maths::cross](#) (`v1`, `v2`, `v3`)

Compute the cross product of two 3D vectors.

- subroutine [maths::update_complex](#) (xc1, xc2, trans, rot_vx, rot_vy, nb_atoms, coords, new_coords)

Apply translation and rotation to a set of atomic coordinates.

- subroutine [maths::vectors_angle_3d](#) (vector1, vector2, theta_degrees)

Compute the angle in degrees between two 3D vectors.

- subroutine [maths::vectors_angle_2d](#) (vector1, vector2, theta_degrees)

Compute the angle in degrees between two vectors projected to 2D.

- subroutine [maths::rmsd](#) (rmsd_value, nb_atoms, coords1, coords2)

Compute the root-mean-square deviation (RMSD) between two sets of coordinates.

- subroutine [maths::calculate_cog](#) (cog, coords, nb_atoms)

Compute the center of geometry (centroid) of a set of points.

- subroutine [maths::calculate_distance](#) (p1, p2, dist)

Compute Euclidean distance between two 3D points.

6.5 mod_assoc.f90 File Reference

Data Types

- type [mod_assoc::type_assoc_file](#)

Data type for storing binned data (eg.

Modules

- module [mod_assoc](#)

Association/complexes file utilities.

Functions/Subroutines

- subroutine [mod_assoc::allocate_assoc_object](#) (this, nlines)

Allocate storage for an association/complexes object.

- integer function [mod_assoc::size_assoc](#) (assoc_file)

Count encounter records in an association/complexes file.

- subroutine [mod_assoc::fill_assoc_object](#) (this, assoc_file)

Read an association/complexes file and populate [type_assoc_file](#).

6.6 mod_clust_algorithm.f90 File Reference

Modules

- module [mod_clust_algorithm](#)

Functions/Subroutines

- subroutine [mod_clust_algorithm::mean_linkage_clustering](#) (matrix, n, output_name, complexes, opt_array)
Perform mean-linkage hierarchical clustering on a distance matrix.
- subroutine [mod_clust_algorithm::sort_complexes](#) (tot_encounters, cluster_indexes, cluster_count, active_clusters, representative_indexes, cluster_average, cluster_sd, array)
Sort clusters and associated arrays by a given key array.
- subroutine [mod_clust_algorithm::write_cluster_elements](#) (tot_encounters, cluster_indexes, cluster_count, active_clusters, output_name)
Write cluster membership lists to a text file.
- subroutine [mod_clust_algorithm::write_clust_info](#) (tot_encounters, representative_indexes, active_clusters, clust_count, cluster_average, cluster_sd, output_name)
Write summary information about clusters to a text file.
- subroutine [mod_clust_algorithm::write_complexes](#) (tot_encounters, cluster_indexes, cluster_count, active_clusters, output_name, complexes)
Write complexes contained in each cluster to individual files.

6.7 mod_matrix.f90 File Reference

Modules

- module [mod_matrix](#)

Functions/Subroutines

- subroutine [mod_matrix::matrix_z_coord](#) (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Build a matrix of absolute differences of Z-coordinates per encounter.
- subroutine [mod_matrix::matrix_atoms_dist](#) (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Build matrix from minimum inter-atomic distances between two solutes.
- subroutine [mod_matrix::matrix_angle](#) (matrix, array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Build matrix of angular differences between encounters.
- subroutine [mod_matrix::matrix_rmsd](#) (matrix, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, coord)
Build RMSD-based pairwise matrix between transformed coordinates.
- subroutine [mod_matrix::write_matrix](#) (matrix, filename)
Write a numeric matrix to a formatted text file.
- subroutine [mod_matrix::write_array](#) (array, filename)
Write a numeric array to a formatted text file.
- subroutine [mod_matrix::read_matrix](#) (matrix, n, filename)
Read a square numeric matrix from a formatted text file.
- subroutine [mod_matrix::read_array](#) (array, n, filename)
Read a numeric array from a formatted text file.

6.8 mod_pdb.f90 File Reference

Data Types

- type `mod_pdb::type_pdb_file`
Data type for storing binned data (eg.
- type `mod_pdb::type_pdb_chain`
- type `mod_pdb::type_pdb_residue`
- type `mod_pdb::type_pdb_atom`

Modules

- module `mod_pdb`
Module to define a pdb type and pdb reading routine for tools.

Functions/Subroutines

- subroutine `mod_pdb::allocate_pdb_object` (this, pdb_file, natoms, nresidues, nchains)
Allocate and initialize a `type_pdb_file` structure from a PDB file.
- integer function `mod_pdb::count_atoms` (pdb_file)
Count ATOM/HETATM records in a PDB file.
- integer function `mod_pdb::count_residues` (pdb_file)
Count residues in a PDB file.
- integer function `mod_pdb::count_chains` (pdb_file)
Count chain separators (TER records) in a PDB file.
- subroutine `mod_pdb::fill_pdb_object` (this, pdb_file)
Parse PDB records and populate the `type_pdb_file` structure.

6.9 mod_threshold.f90 File Reference

Modules

- module `mod_threshold`

Functions/Subroutines

- subroutine `mod_threshold::array_z_coord` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter.
- subroutine `mod_threshold::array_atoms_dist` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1 ↔ _crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter.
- subroutine `mod_threshold::array_angle` (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter.
- subroutine `mod_threshold::sort_array` (arr, encounter_indexes)
Sort an array in ascending order and reorder encounter indexes accordingly.

6.10 read_input.f90 File Reference

Modules

- module [read_input](#)

Functions/Subroutines

- subroutine [read_input::read_pdb](#) (pdb, filename, tot_atoms, tot_residues, tot_chains)
Read a PDB file into a `type_pdb_file` object.
- subroutine [read_input::read_assoc](#) (assoc, filename, tot_encounters)
Read an association/complexes file into a `type_assoc_file` object.

6.11 threshold.f90 File Reference

Functions/Subroutines

- program [main](#)
Generate encounter selection based on threshold criteria.
- subroutine [print_help](#) (help_option)
Print usage/help text for `threshold`.
- subroutine [read_atoms_coord](#) (atoms_array, atoms_coord, tot_atoms, pdb, pdb_filename)
Read coordinates for a specified list of atoms (names or indices).
- subroutine [write_complexes](#) (complexes_arg, distarray_arg, encounter_indexes_arg, nb_encounters_arg, cutoff_arg, output_name_arg)
Write selected encounter complexes to an output file.
- subroutine [write_array](#) (array_arg, filename_arg, cutoff_arg)
Write array values up to a cutoff to a formatted file.

6.11.1 Function/Subroutine Documentation

6.11.1.1 main()

```
program main
```

Generate encounter selection based on threshold criteria.

This executable computes thresholds (`z_coord`, `atoms_dist`, `angles`) across encounter transforms and writes selected encounter records to an output complexes file. See [print_help](#) for usage examples.

6.11.1.2 print_help()

```
subroutine main::print_help (
    character(len=*), intent(in) help_option )
```

Print usage/help text for threshold.

Explains command-line options and provides examples per threshold type.

6.11.1.3 read_atoms_coord()

```
subroutine main::read_atoms_coord (
    character(len=*), dimension(:), intent(in) atoms_array,
    real(kind=8), dimension(:, :), intent(out), allocatable atoms_coord,
    integer, intent(in) tot_atoms,
    type ( type_pdb_file ), intent(in) pdb,
    character(len=*), intent(in) pdb_filename )
```

Read coordinates for a specified list of atoms (names or indices).

Similar to the helper in `make_matrix`. Returns an allocated `atoms_coord` array with matching coordinates for the atom names or indices provided in `atoms_array`.

6.11.1.4 write_array()

```
subroutine main::write_array (
    real(kind=8), dimension(:), intent(in) array_arg,
    character*128, intent(in) filename_arg,
    real(kind=8), intent(in) cutoff_arg )
```

Write array values up to a cutoff to a formatted file.

Writes values from `array` (in ascending order) until `cutoff` is reached and stores them in `filename` using `F10.4` formatting.

6.11.1.5 write_complexes()

```
subroutine main::write_complexes (
    type(type_assoc_file), intent(in) complexes_arg,
    real (kind=8), dimension(:), intent(in) distarray_arg,
    integer, dimension(:), intent(in) encounter_indexes_arg,
    integer, intent(in) nb_encounters_arg,
    real(kind=8), intent(in) cutoff_arg,
    character*128, intent(in) output_name_arg )
```

Write selected encounter complexes to an output file.

Writes the header lines from `complexeshead` and then appends encounter lines (from `complexeslines`) in the order provided by `encounter_indexes` until a value exceeding `cutoff` is encountered.

Index

- allocate_assoc_object
 - mod_assoc, 11
- allocate_pdb_object
 - mod_pdb, 22
- analyze_residues.f90, 37
 - main, 37
 - print_help, 37
- array_angle
 - mod_threshold, 24
- array_atoms_dist
 - mod_threshold, 25
- array_z_coord
 - mod_threshold, 25
- atoms
 - mod_pdb::type_pdb_file, 33
 - mod_pdb::type_pdb_residue, 34
- beta
 - mod_pdb::type_pdb_atom, 31
- calculate_cog
 - maths, 7
- calculate_distance
 - maths, 8
- chainid
 - mod_pdb::type_pdb_atom, 31
 - mod_pdb::type_pdb_residue, 35
- chains
 - mod_pdb::type_pdb_file, 33
- clust.f90, 38
 - main, 38
 - print_help, 38
- coord
 - mod_pdb::type_pdb_atom, 31
- count_atoms
 - mod_pdb, 22
- count_chains
 - mod_pdb, 22
- count_residues
 - mod_pdb, 23
- cross
 - maths, 8
- fill_assoc_object
 - mod_assoc, 11
- fill_pdb_object
 - mod_pdb, 23
- head
 - mod_assoc::type_assoc_file, 29
- id
 - mod_pdb::type_pdb_atom, 31
- lines
 - mod_assoc::type_assoc_file, 30
- main
 - analyze_residues.f90, 37
 - clust.f90, 38
 - make_matrix.f90, 38
 - threshold.f90, 43
- make_matrix.f90, 38
 - main, 38
 - print_help, 39
 - read_atoms_coord, 39
- maths, 7
 - calculate_cog, 7
 - calculate_distance, 8
 - cross, 8
 - rmsd, 8
 - update_complex, 9
 - vectors_angle_2d, 9
 - vectors_angle_3d, 10
- maths.f90, 39
- matrix_angle
 - mod_matrix, 16
- matrix_atoms_dist
 - mod_matrix, 17
- matrix_rmsd
 - mod_matrix, 17
- matrix_z_coord
 - mod_matrix, 18
- mean_linkage_clustering
 - mod_clust_algorithm, 13
- mod_assoc, 10
 - allocate_assoc_object, 11
 - fill_assoc_object, 11
 - size_assoc, 12
- mod_assoc.f90, 40
- mod_assoc::type_assoc_file, 29
 - head, 29
 - lines, 30
 - nlines, 30
 - rot1, 30
 - rot2, 30
 - trans_vector, 30
 - xc1, 30
 - xc2, 30
- mod_clust_algorithm, 12
 - mean_linkage_clustering, 13

- sort_complexes, 13
- write_clust_info, 14
- write_cluster_elements, 14
- write_complexes, 15
- mod_clust_algorithm.f90, 40
- mod_matrix, 15
 - matrix_angle, 16
 - matrix_atoms_dist, 17
 - matrix_rmsd, 17
 - matrix_z_coord, 18
 - read_array, 18
 - read_matrix, 20
 - write_array, 20
 - write_matrix, 20
- mod_matrix.f90, 41
- mod_pdb, 21
 - allocate_pdb_object, 22
 - count_atoms, 22
 - count_chains, 22
 - count_residues, 23
 - fill_pdb_object, 23
- mod_pdb.f90, 42
- mod_pdb::type_pdb_atom, 31
 - beta, 31
 - chainid, 31
 - coord, 31
 - id, 31
 - name, 32
 - occ, 32
 - record, 32
 - resid, 32
 - resname, 32
- mod_pdb::type_pdb_chain, 32
- mod_pdb::type_pdb_file, 33
 - atoms, 33
 - chains, 33
 - natoms, 33
 - nchains, 33
 - nresidues, 34
 - residues, 34
- mod_pdb::type_pdb_residue, 34
 - atoms, 34
 - chainid, 35
 - natoms, 35
 - resid, 35
 - resname, 35
- mod_threshold, 24
 - array_angle, 24
 - array_atoms_dist, 25
 - array_z_coord, 25
 - sort_array, 26
- mod_threshold.f90, 42
- name
 - mod_pdb::type_pdb_atom, 32
- natoms
 - mod_pdb::type_pdb_file, 33
 - mod_pdb::type_pdb_residue, 35
- nchains
 - mod_pdb::type_pdb_file, 33
- nlines
 - mod_assoc::type_assoc_file, 30
- nresidues
 - mod_pdb::type_pdb_file, 34
- occ
 - mod_pdb::type_pdb_atom, 32
- print_help
 - analyze_residues.f90, 37
 - clust.f90, 38
 - make_matrix.f90, 39
 - threshold.f90, 43
- read_array
 - mod_matrix, 18
- read_assoc
 - read_input, 26
- read_atoms_coord
 - make_matrix.f90, 39
 - threshold.f90, 44
- read_input, 26
 - read_assoc, 26
 - read_pdb, 27
- read_input.f90, 43
- read_matrix
 - mod_matrix, 20
- read_pdb
 - read_input, 27
- record
 - mod_pdb::type_pdb_atom, 32
- resid
 - mod_pdb::type_pdb_atom, 32
 - mod_pdb::type_pdb_residue, 35
- residues
 - mod_pdb::type_pdb_file, 34
- resname
 - mod_pdb::type_pdb_atom, 32
 - mod_pdb::type_pdb_residue, 35
- rmsd
 - maths, 8
- rot1
 - mod_assoc::type_assoc_file, 30
- rot2
 - mod_assoc::type_assoc_file, 30
- size_assoc
 - mod_assoc, 12
- sort_array
 - mod_threshold, 26
- sort_complexes
 - mod_clust_algorithm, 13
- threshold.f90, 43
 - main, 43
 - print_help, 43
 - read_atoms_coord, 44
 - write_array, 44

- write_complexes, [44](#)
- trans_vector
 - mod_assoc::type_assoc_file, [30](#)
- update_complex
 - maths, [9](#)
- vectors_angle_2d
 - maths, [9](#)
- vectors_angle_3d
 - maths, [10](#)
- write_array
 - mod_matrix, [20](#)
 - threshold.f90, [44](#)
- write_clust_info
 - mod_clust_algorithm, [14](#)
- write_cluster_elements
 - mod_clust_algorithm, [14](#)
- write_complexes
 - mod_clust_algorithm, [15](#)
 - threshold.f90, [44](#)
- write_matrix
 - mod_matrix, [20](#)
- xc1
 - mod_assoc::type_assoc_file, [30](#)
- xc2
 - mod_assoc::type_assoc_file, [30](#)